

THEORY OF ARTIFICIAL INTELLIGENCE

Lecture 1 — Foundations

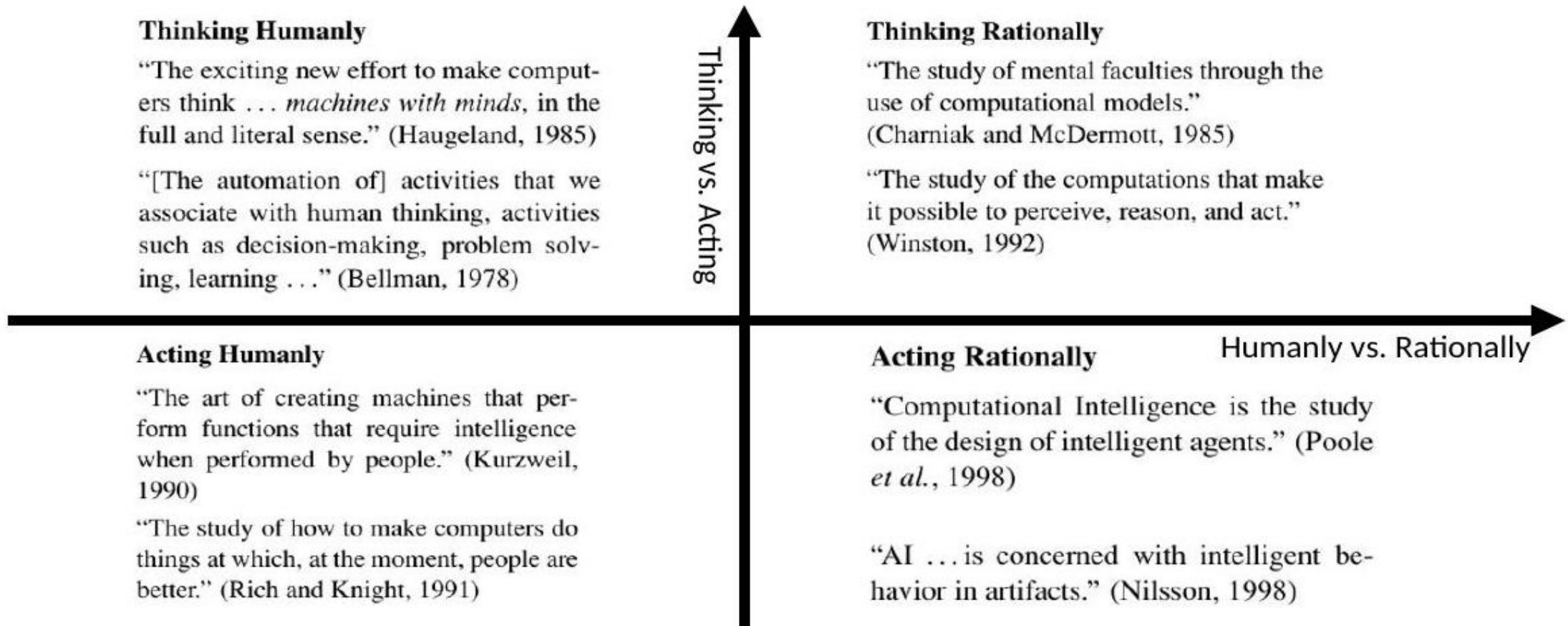
History, Course Structure, and the Language of Agents

Three Machines, Three Questions

- A chess engine picks a move in milliseconds — is it “thinking”?
- A self-driving car brakes for a pedestrian it has never seen before — how did it decide that was the right action?
- A language model drafts an email, then books your flight — where does “assistant” end and “agent” begin?
- **By the end of this course, you'll have precise vocabulary and formal tools for all three questions.**

What Do We Mean by “AI”?

The Four Quadrants



Source: *Artificial Intelligence: A Modern Approach* — Stuart J. Russell and Peter Norvig

What Are These Four Definitions For?

Thinking Humanly

Success = Cognitive Plausibility

Does it match how humans actually think?

Thinking Rationally

Success = Logical Correctness

Is the reasoning valid and sound?

Acting Humanly

Success = Indistinguishability

Can a human tell the difference?

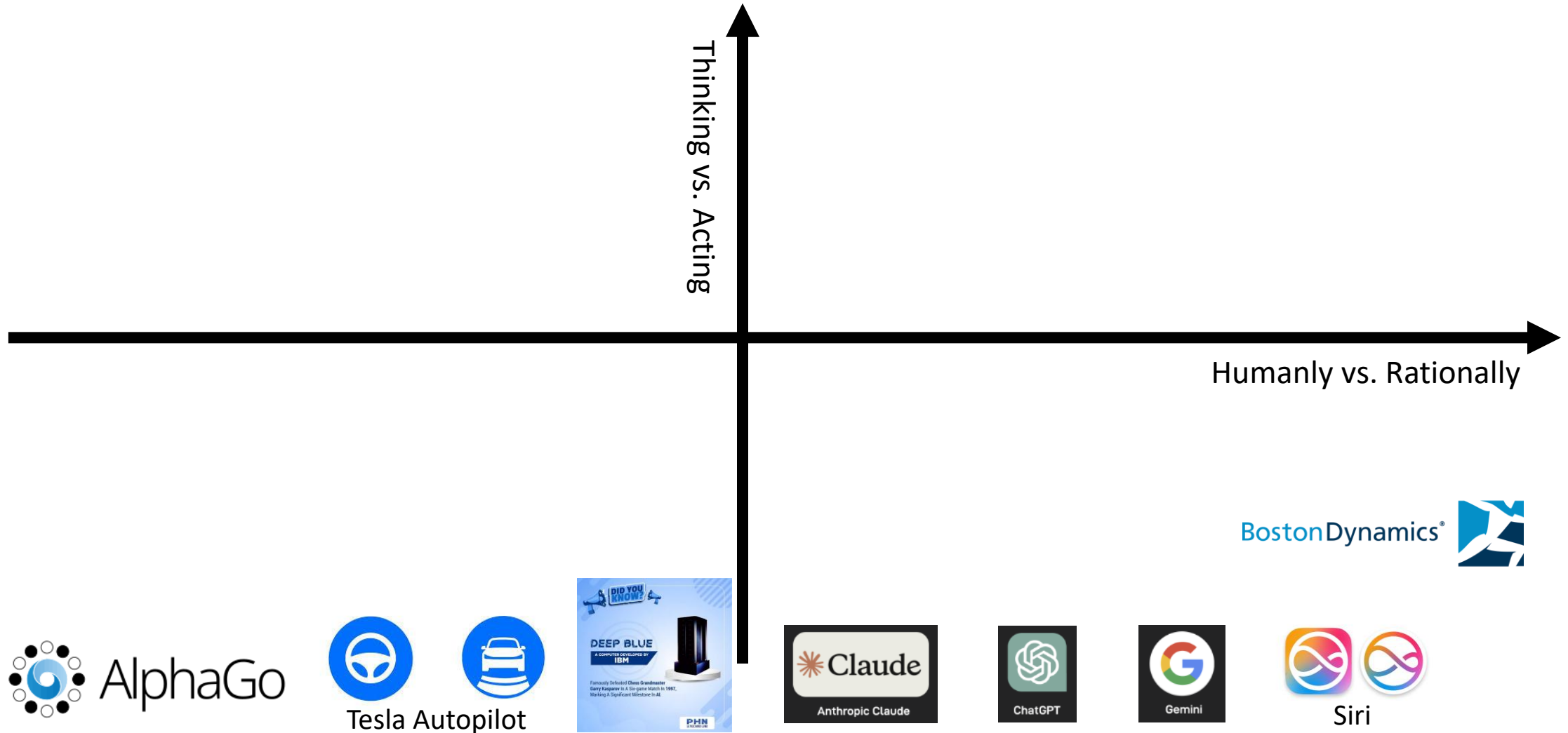
Acting Rationally

Success = Goal Optimality

Does it maximize the expected outcome?

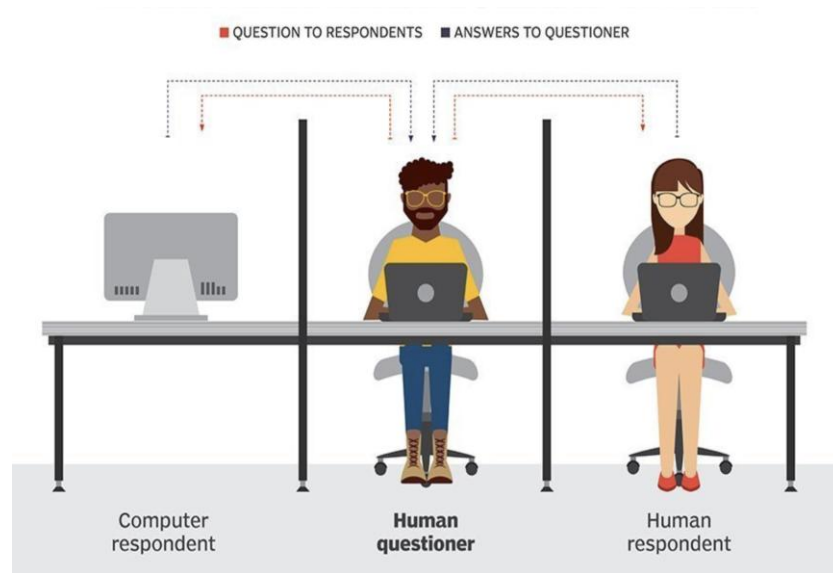
Not product categories, but ways of thinking about intelligence.

Quick Exercise: Where Do They Fit?

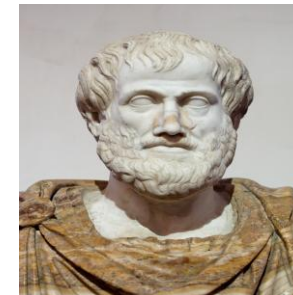


What Do We Mean by “AI”?

- Acting humanly — passing for a human (the Turing Test tradition)



- Thinking humanly — modeling human cognition (cognitive science tradition)
- Thinking rationally — formal logic and correct inference (the “laws of thought”)
- Acting rationally — the rational-agent view: choosing actions that best achieve goals given what's known**
- This course is organized around the rational-agent view — it's the thread that connects search, logic, RL, and agentic AI



PART 1

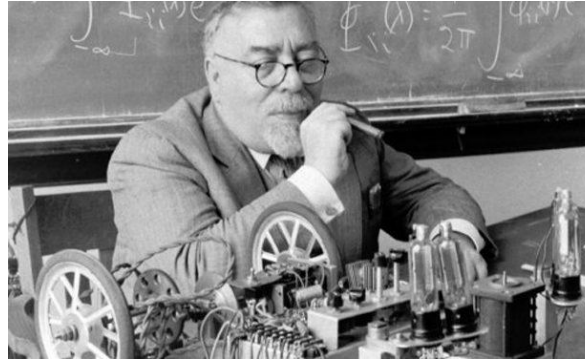
A Brief History of AI

From symbolic optimism to statistical learning to agentic systems

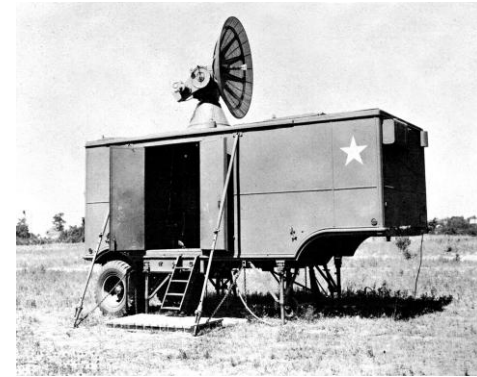
The Foundations



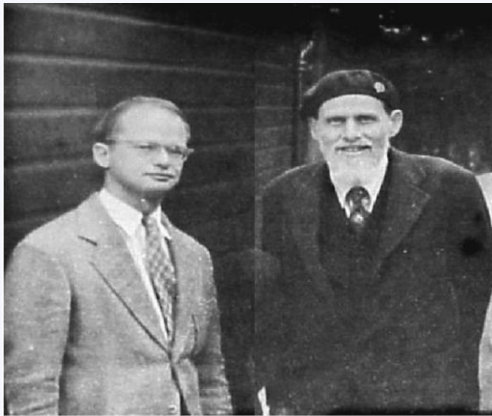
Norbert Wiener



The SCR-584 system



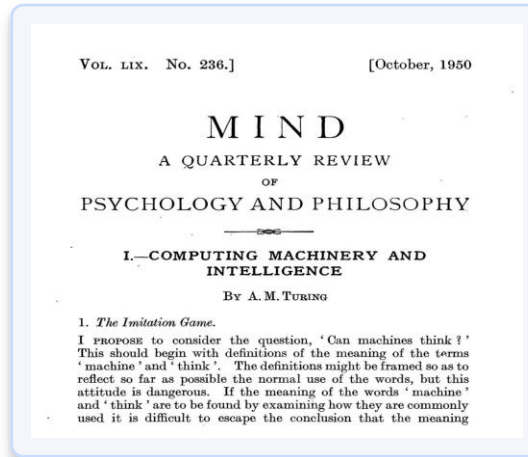
1944 <https://youtu.be/i5Vq3wYPXU>



1943

McCulloch & Pitts

Boolean circuit model of the brain — brain as a machine.



1950

Turing

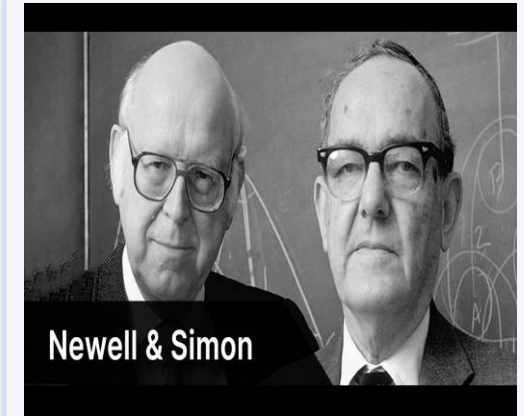
“Computing Machinery and Intelligence” —
“Can machines think?”



1952

Samuel

Checkers-playing program — the first machine that “learns.”



1955

Newell & Simon

Logic Theorist — machine as a mathematician.

The Birth of AI: 1956 Dartmouth Conference

- Duration: over a month
- Attendance: only about 10 people — but all “giants”
- Key figures: John McCarthy, Marvin Minsky, Claude Shannon, and others
- **The core belief: every aspect of learning or any other feature of intelligence can be so precisely described that a machine can be made to simulate it**



Origins: Turing and Dartmouth

1950–56

- Alan Turing's 1950 paper “Computing Machinery and Intelligence” proposes the imitation game — later known as the Turing Test
- 1956 Dartmouth Workshop: McCarthy, Minsky, Rochester, and Shannon coin the term “artificial intelligence”
- Founding claim: every aspect of learning or intelligence can in principle be so precisely described that a machine can simulate it
- This is the birth of AI as an academic field, not just a philosophical question

Early Programs and Symbolic AI

1956–69

- Logic Theorist (1956) and General Problem Solver (1957) — early programs that prove theorems and solve puzzles using symbolic search
- Perceptrons (Rosenblatt, 1958) — an early trainable, brain-inspired classifier
- Minsky & Papert's Perceptrons (1969) proves single-layer perceptrons can't represent simple functions (e.g. XOR)
- This result — combined with unmet promises elsewhere — cools enthusiasm for both symbolic AI and neural approaches

The First AI Winter

1974–80

- Early systems didn't scale — combinatorial explosion in search, brittleness outside narrow toy domains
- The 1973 Lighthill Report to the UK government concludes AI has failed to deliver on its promises
- US and UK funding agencies (DARPA, research councils) sharply cut AI research funding
- Lesson for the field: matching claims to demonstrated capability matters as much as the ideas themselves

Expert Systems: The Second Wave

1980–87

- Rule-based systems encode a human expert's knowledge as if-then rules (e.g. MYCIN for medical diagnosis, DENDRAL for chemistry, XCON for computer configuration)
- Industry adopts expert systems commercially — XCON reportedly saves DEC tens of millions of dollars a year
- Knowledge engineering becomes a discipline of its own: extracting expert rules is the bottleneck, not the reasoning engine
- This is the direct ancestor of the Expert Systems unit later in the course

Expert Systems Bust, Second Winter

1987–93

- Rule-based systems are brittle: they fail ungracefully outside the exact scenarios their rules anticipated
- Maintaining and scaling hand-written rule bases becomes unmanageable as systems grow
- The specialized LISP-machine hardware market collapses as general-purpose workstations catch up
- Funding contracts again — the term “AI” itself becomes commercially unfashionable through the 1990s

The Statistical Turn

1990s–2000s

- Focus shifts from hand-coded rules to learning patterns directly from data — probability and statistics become central
- Machine learning (decision trees, SVMs, Bayesian methods, early neural nets) starts outperforming hand-engineered systems
- 1997: IBM's Deep Blue defeats world chess champion Garry Kasparov — brute-force search plus evaluation functions, not learning
- AI re-enters industry quietly, often rebranded as “machine learning” or “data mining” rather than “AI”

Deep Learning and Deep RL

2006–17

- 2012: AlexNet wins the ImageNet competition by a huge margin using deep convolutional networks on GPUs — the moment deep learning becomes impossible to ignore
- Cheap parallel compute (GPUs) plus large labeled datasets finally make deep networks practical, not just theoretical
- 2013–16: Deep reinforcement learning milestones — Atari from raw pixels, then AlphaGo defeats Lee Sedol at Go in 2016
- Investment and enthusiasm return at a scale the field hadn't seen since the 1980s

Foundation Models and Agentic AI

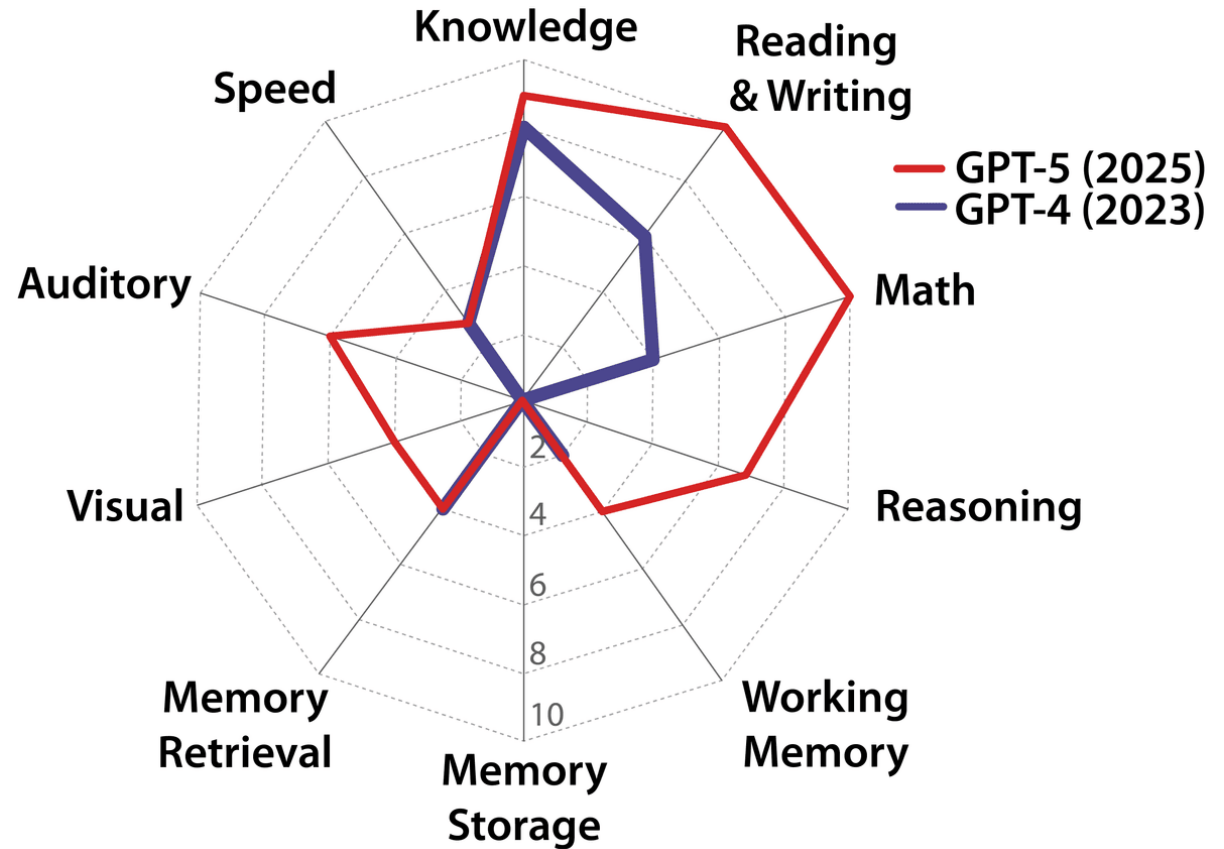
2018–now

- Large language models trained on internet-scale text (GPT-family, and successors) show broad, general-purpose capability from a single model
- Reinforcement learning from human feedback (RLHF) aligns model outputs with human preferences and instructions
- Models gain the ability to call tools, browse, write and execute code, and take multi-step actions toward a goal — “agentic AI”
- This is where the field stands today, and where this course's final unit will leave you

Foundation Models and Agentic AI on the Way to GenAI

future

The original clause (reported Dec 2024):
Microsoft and OpenAI's contract reportedly defined AGI not by any technical capability test, but as the point when OpenAI's AI systems generate **at least \$100 billion in profit**.
Per one detailed account, the more precise figure from the underlying 2023 investment-round documents was closer to **\$92 billion**



<https://www.agidefinition.ai>

PART 2

Course Map

Five units, each the field's best answer at a moment in its history

The Five Units, In Historical Order

01

Search

Classical problem-solving:
uninformed, informed, local &
adversarial search

← 1950s–60s

02

Logic & Knowledge Rep.

Formal reasoning and
representing what an agent
knows

← 1960s–70s

03

Expert Systems

Encoding expert rules for
narrow, high-value domains

← 1980s

04

Reinforcement Learning

Learning behavior from reward,
from Atari to RLHF

← 1990s–today

05

Agentic AI

Models that plan, call tools, and
act toward goals

← today

The Five Units, In Historical Order and Taxonomy

01

Search

Classical problem-solving:
uninformed, informed, local &
adversarial search

← 1950s–60s

03

Expert Systems

Encoding expert rules for
narrow, high-value domains

← 1980s

02

Logic & Knowledge Rep.

Formal reasoning and
representing what an agent
knows

← 1960s–70s

04

Reinforcement Learning

Learning behavior from reward,
from Atari to RLHF

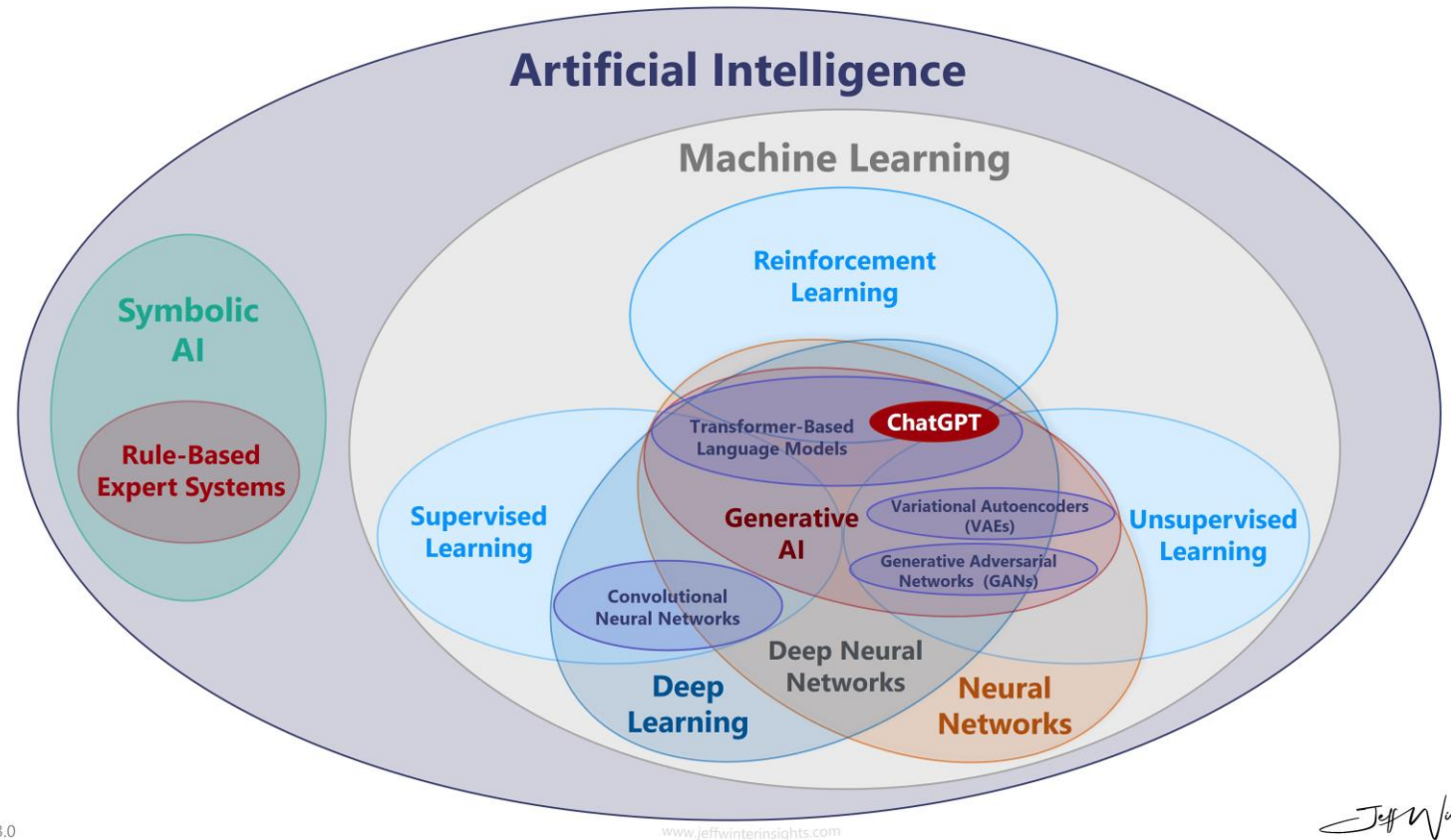
← 1990s–today

05

Agentic AI

Models that plan, call tools, and
act toward goals

← today



Rev 3.0

www.jeffwinterinsights.com

Jeff Winter

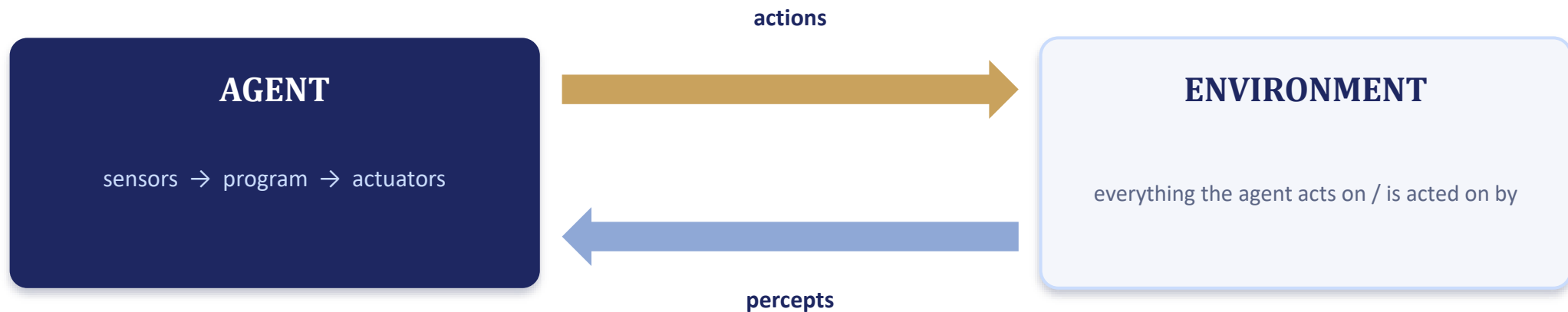
PART 3

Agents and Environments

The vocabulary you'll use in every remaining lecture

What Is an Agent?

An agent is anything that perceives its environment through sensors and acts upon that environment through actuators.



Percept/Observation: the agent's perceptual input at a given instant.

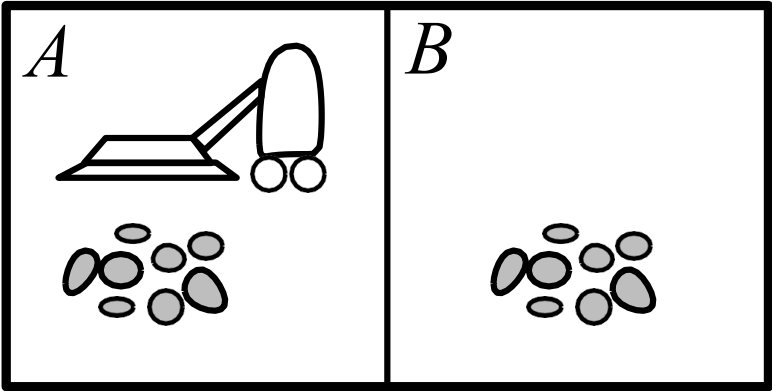
Percept sequence: the complete history of everything the agent has ever perceived.

Rational agent: for each possible percept sequence, selects an action expected to maximize its performance measure, given what it knows.

What Is “State”?

- World state — the complete, true configuration of the environment at a given instant (e.g. which vacuum-world squares are dirty, and where the agent is)
- Percept / belief state — what the agent itself can currently perceive or infer, which may be a partial or noisy view of the world state
- **These two coincide only when the environment is fully observable — a distinction we formalize next**
- State is the object search algorithms operate over later in the course: a “state space” is just the set of all world states reachable by the agent's actions

A Minimal Example: The Cleaner Robot (Vacuum-Cleaner World)



Environment:

Two locations: A and B
Each can be clean or dirty

Percepts:

Current location
Whether the current square is dirty

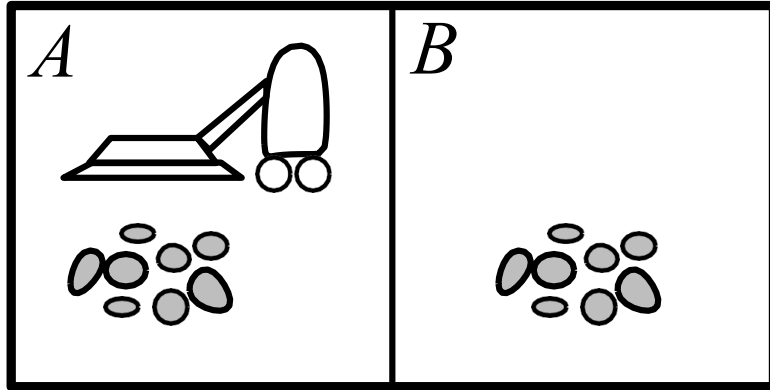
Actions:

Left, Right, Clean

Percept/Observed Sequence	Action
(A, Dirty)	Clean
(A, Dirty), (A, Clean)	Right
(A, Dirty), (A, Clean), (B, Dirty)	Clean
(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)	Left
...	...

Source: Artificial Intelligence: A Modern Approach — Stuart J. Russell and Peter Norvig

Agent Function (Vacuum-Cleaner World)



Environment:

Two locations: A and B

Each can be clean or dirty

Percepts:

Current location

Whether the current square is dirty

$O = \{(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)\}$

Actions:

$A = \{Left, Right, Clean\}$

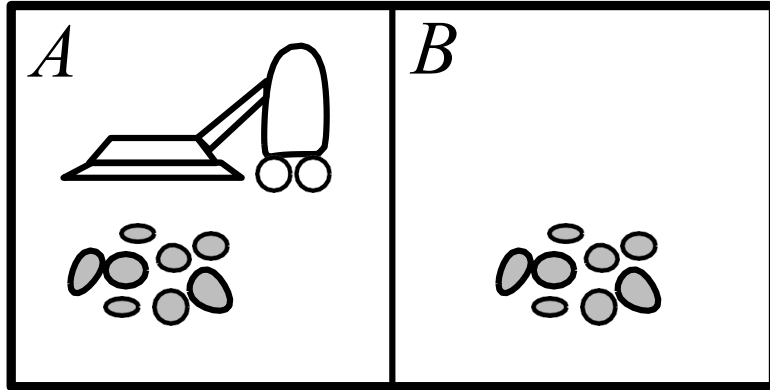
Percept/Observed Sequence	Action
(A, Dirty)	Clean
(A, Dirty), (A, Clean)	Right
(A, Dirty), (A, Clean), (B, Dirty)	Clean
(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)	Left
...	...

Agent function maps a percepts sequence to the action:

$$AF: O \times O \times \dots \times O \rightarrow A$$

$$o_0, o_1, \dots, o_n \mapsto a_n$$

Table Agent Function VS Agent's Program (Vacuum-Cleaner World)



Environment:

Two locations: A and B
Each can be clean or dirty

Percepts:

Current location
Whether the current square is dirty
 $O = \{(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)\}$

Actions:

$A = \{Left, Right, Clean\}$

Percept/Observed Sequence	Action
(A, Dirty)	Clean
(A, Dirty), (A, Clean)	Right
(A, Dirty), (A, Clean), (B, Dirty)	Clean
(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)	Left
...	...

Agent function maps a percepts sequence to the action:

$$AF: O \times O \times \dots \times O \rightarrow A$$

$$o_0, o_1, \dots, o_n \mapsto a_n$$

agent_program(o_n):

if $o_n[1] == Dirty$:

return *Clean*

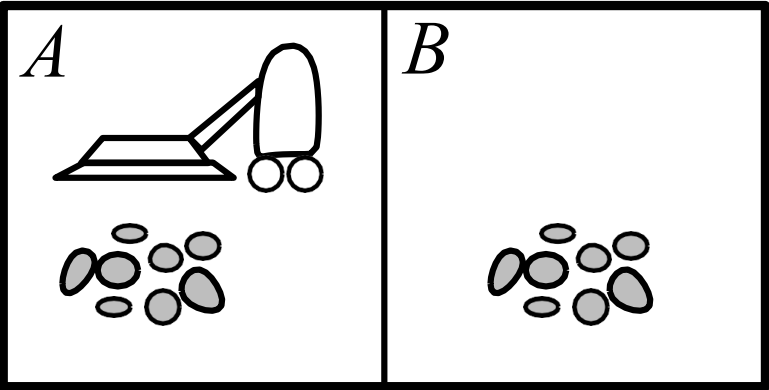
elif $o_n[0] == A$:

return *Right*

else:

return *Left*

State VS Percept (Vacuum-Cleaner World)



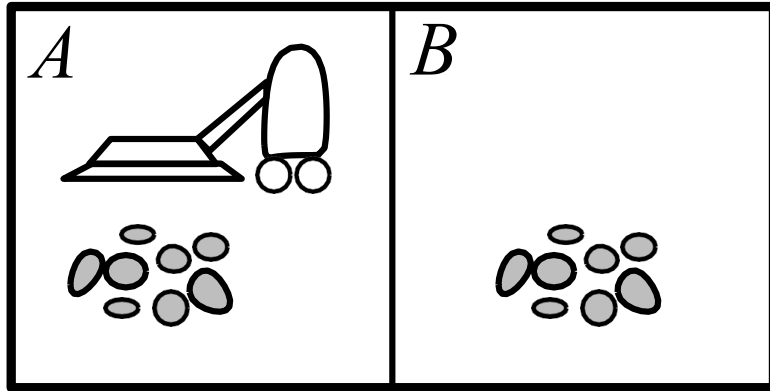
Environment:
Two locations: A and B
Each can be clean or dirty

Percepts:
Current location
Whether the current square is dirty
 $O = \{(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)\}$

Actions:
 $A = \{Left, Right, Clean\}$

Percept/Observed Sequence	State (agent in, A, B)	Action
(A, Dirty)	(A, Dirty, Dirty)	Clean
(A, Dirty), (A, Clean)	(A, Clean, Dirty)	Right
(A, Dirty), (A, Clean), (B, Dirty)	(B, Clean, Dirty)	Clean
(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)	(B, Clean, Clean)	Left
...	...	...

Transition Model (Vacuum-Cleaner World)



Environment:

Two locations: A and B

Each can be clean or dirty

Percepts:

Current location

Whether the current square is dirty

$$O = \{(A, \text{Dirty}), (A, \text{Clean}), (B, \text{Dirty}), (B, \text{Clean})\}$$

Actions:

$$A = \{\text{Left}, \text{Right}, \text{Clean}\}$$

Percept/Observed Sequence	State (agent in, A, B)	Action
(A, Dirty)	(A, Dirty, Dirty)	Clean
(A, Dirty), (A, Clean)	(A, Clean, Dirty)	Right
(A, Dirty), (A, Clean), (B, Dirty)	(B, Clean, Dirty)	Clean
(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)	(B, Clean, Clean)	Left
...	...	...

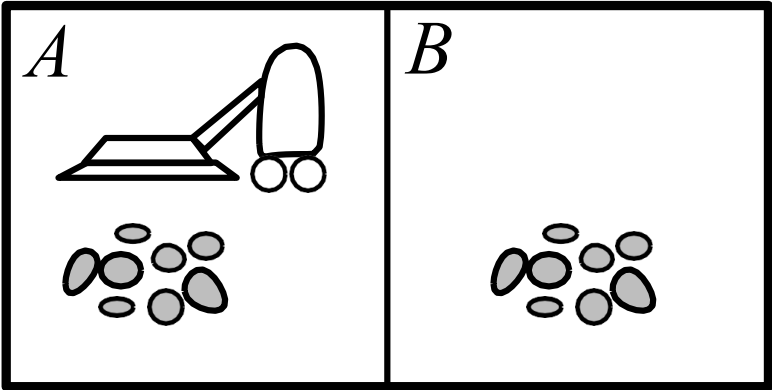
Transition model maps the current state and action to a new state:

$$TM: S \times A \rightarrow S$$

$$(s, a) \mapsto s'$$

For example, $((A, \text{Dirty}, \text{Dirty}), \text{Right}) \mapsto (A, \text{Dirty}, \text{Dirty})$

State Space (Vacuum-Cleaner World)



Environment:

Two locations: A and B
Each can be clean or dirty

Percepts:

Current location
Whether the current square is dirty
 $O = \{(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)\}$

Actions:

$A = \{Left, Right, Clean\}$

Percept/Observed Sequence	State (agent in, A, B)	Action
(A, Dirty)	(A, Dirty, Dirty)	Clean
(A, Dirty), (A, Clean)	(A, Clean, Dirty)	Right
(A, Dirty), (A, Clean), (B, Dirty)	(B, Clean, Dirty)	Clean
(A, Dirty), (A, Clean), (B, Dirty), (B, Clean)	(B, Clean, Clean)	Left
...	...	...

Transition model maps the current state and action to a new state:

$$TM: S \times A \rightarrow S$$
$$(s, a) \mapsto s'$$

If we start from the state s_0 and apply recursively transition model with all possible actions, we get the set of reachable states, which is called **state space**.

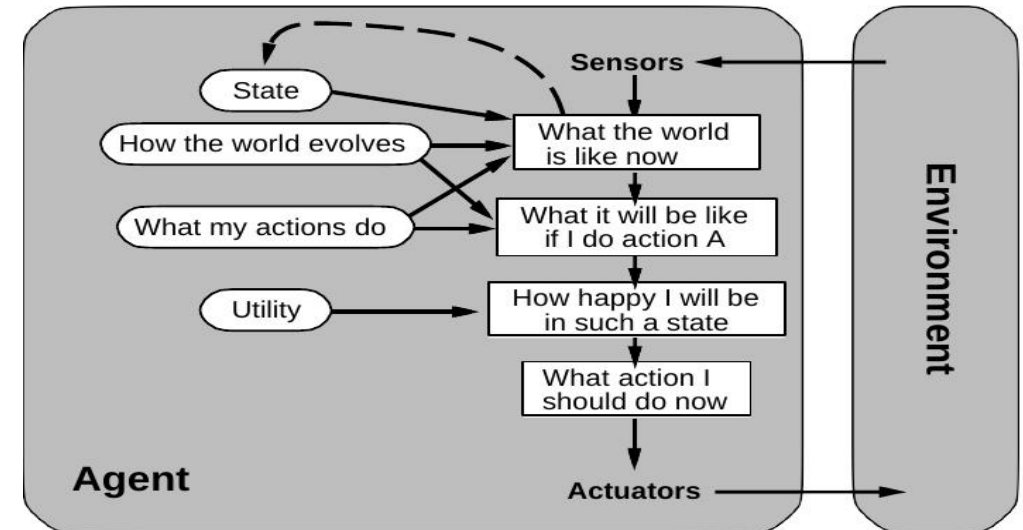
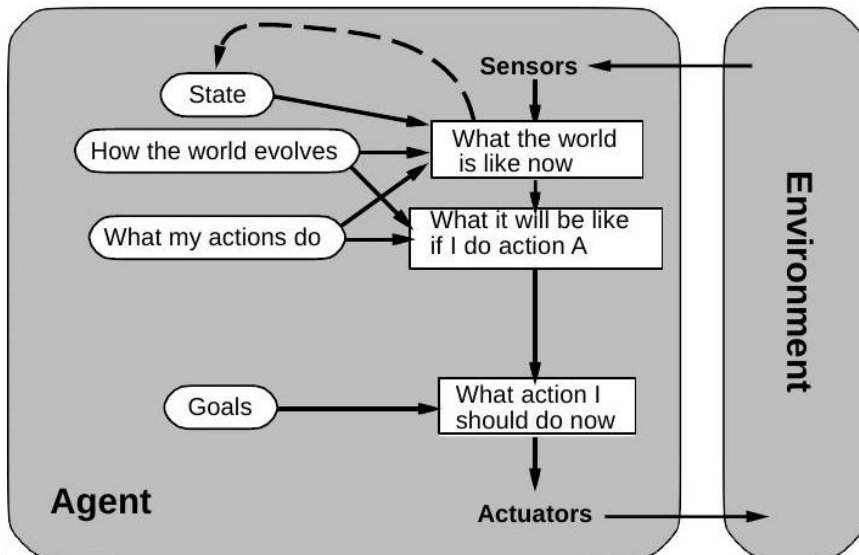
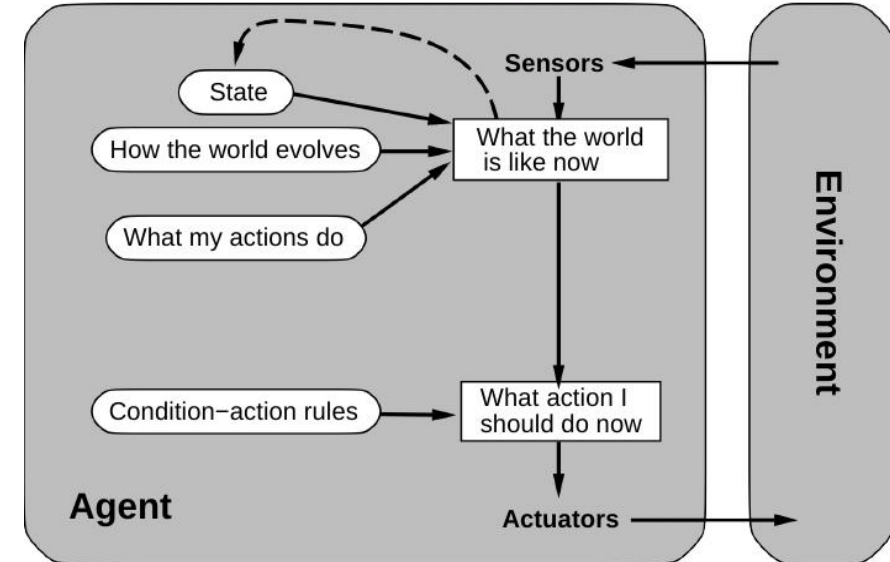
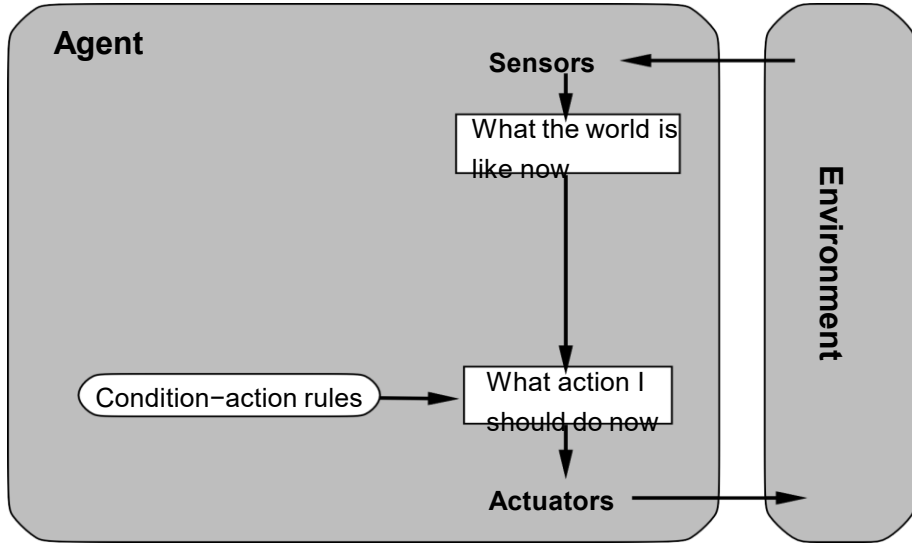
Specifying a Task: PEAS

- **Performance measure** — how do we judge success? (defined externally to the agent, not by the agent itself)
- **Environment** — what world does the agent operate in?
- **Actuators** — how does the agent act on the environment?
- **Sensors** — how does the agent perceive the environment?
- PEAS is the first thing to nail down for any agent design problem — get it wrong and everything downstream (search, planning, learning) optimizes the wrong thing

Example: Vacuum-Cleaner World

P	E	A	S
Amount of dirt cleaned per time; movement cost	Two locations, each clean/dirty	Wheels (Left/Right), suction unit (Clean)	Location sensor, dirt sensor

Simple Reflex/Model-Based Reflex/Goal-Based/Utility-Based Agents



PEAS in Practice: Self-Driving Car

Performance

Safety, legal-lane compliance, comfort, trip time, fuel/energy efficiency

Environment

Roads, other traffic, pedestrians, signage, weather, other drivers

Actuators

Steering, accelerator, brake, signal, horn, display

Sensors

Cameras, LIDAR/radar, GPS, speedometer, odometer, accelerometer

PEAS in Practice: Vacuum World

Performance

Amount of dirt cleaned per unit time, energy used, noise generated

Environment

A set of discrete locations (e.g. two rooms), each either clean or dirty

Actuators

Move left, move right, suck

Sensors

Location sensor, dirt sensor (is the current square dirty?)

This deliberately tiny, toy environment is the running example we'll reuse across the taxonomy slides that follow.

Classifying Environments

- Not all environments are equally hard — the right algorithm for an agent depends entirely on the kind of environment it faces
- Russell & Norvig give seven dimensions along which any environment can be classified
- Observable, deterministic, episodic, static, discrete, single/multi-agent, and known — we'll take each in turn, using the vacuum world and a few richer examples
- **By the end, you should be able to classify a new environment along all seven axes in under a minute**

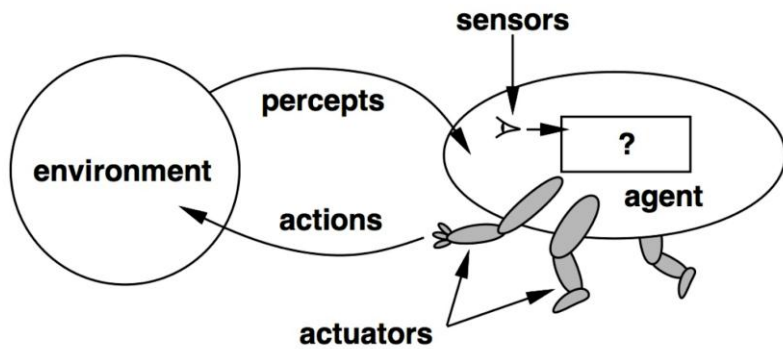
The Seven Dimensions, At a Glance

Dimension	Simpler side	Harder side
Observable	Fully	Partially
Determinism	Deterministic	Stochastic
Episodes	Episodic	Sequential
Time	Static	Dynamic
Values	Discrete	Continuous
Agents	Single-agent	Multi-agent
Rules	Known	Unknown

Classifying Environments (Example)

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete
Figure 2.6 Examples of task environments and their characteristics.						

Agent Structure



An agent can be viewed as two parts:

- Architecture: the physical hardware, sensors, and actuators.
- Program: the algorithm that maps percepts to actions.

Formally: $\text{agent} = \text{architecture} + \text{program}$

In this course, we will mostly focus on how to design the agent program.